

SPRINGBOOT

Existe varias maneras de programar en springboot lo hare en IntelliJ pero puedes hacerlo en programación. podemos utilizar en tipo Gradle - Groovy.

Yo suelo utilizar Marven

Las dependencias

de momento solo spring web

Primer run

Hemos corrido el proyecto y ha funcionado, devuelve un fallo que dira que si funciona

vale, para empezar a tope vamos a añadir el primero Hola mundo, aqui prendemos lo que es **get** que sera el que nos va acompañar a lo largo de clases.

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class DemoApplication {

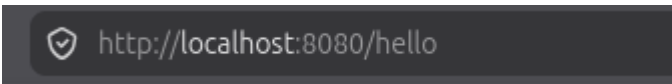
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    // EL PRIMERO GET DE TODO EL MUNDO DE SPRINGBOOT

    @GetMapping("/hello")
    public String hello(@RequestParam(value = "name",defaultValue =
"World") String name) {
        return String.format("Hello %s",name);
    }
}
```

y al ejecutar la url de **/hello** tendremos nuestro primer contacto con backend

lo que tienes en la URL es:

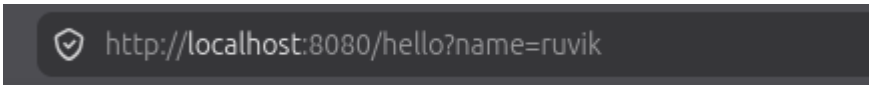
A dark-themed browser address bar with a shield icon on the left and the text "http://localhost:8080/hello" in white.

Y el resultado tiene que se el siguiente:

Hello World

al efectuar ahora la URL de `/hello?name=ruvik` esto ahora con la logica de sprinboot debde saludar al **ruvik**

lo que tienes en la URL es:



Y el resultado tiene que se el siguiente:

Hello ruvik

Fallos

Como no me acordaba como funcionaba, confundía que `get` y `post` iba de la mano, pero estaba equivocada, primero es un `get`, donde uno va poder mostrar el formulario.

de momento esto falla debo averiguar porque falla.

Realmente tuve que investigar, porque tire todo de memoria, me confundi por ejemplo del `<form method="POST" action="/formulario" enctype="multipart/form-data">` porque se me olvido que `action` va directamente buscando `/formulari` y yo en `@PostMapping(/mostrar)` tenia puesto otra URL

[!IMPORTANT]

IMPORTANTE: recuerda siempre lo que este en `<form method="POST" action="/formulario" enctype="multipart/form-data">` siempre el `post` que lo precesa debe tener el mismo nombre.

Y RECUERDA QUE EL `NAME` SIEMPRE VA IR DE LA MANO CON EL `@RequestParam` CON EL MISMO NOMBRE

Solución

```
@GetMapping(value = "/")
public String mostrarFormulario(){
    return "/formulario.html" ;
}
```

en la URL `/formulario.html` pedimos el fichero html

en el `post` si procesa y en ese mismo procesamiento podemos mostrar los datos directamente, pero primero debemos hacer un html:

```
<html>
  <body>
    <form method="POST" action="/formulario" enctype="multipart/form-data">
      <label for="nombre">Nombre:</label>
      <input type="text" name="nombre" id="nombre">

      <button type="submit">Enviar</button>
    </form>
  </body>
</html>
```

Y el `post` mostramos lo siguiente:

```
@PostMapping("/formulario")
public String procesarDatos(@RequestParam(value = "nombre") String
nombre){
    System.out.println("Procesando proceso de datos");

    if (!nombre.isEmpty()){
        return String.format("Hola %s", nombre) ;
    }
    return String.format("Hola mundo") ;
}
```

Recogemos lo que es el valor de `type` del `input` por medio del `@RequestParam` y luego lo mostramos directamente en el `@PostMapping`.